

TOMBOT POKEMON TRACKER™

Aplicación web personal para gestionar, visualizar y monitorizar una colección física de cartas Pokémon by TomBot.

La aplicación mantiene un **catálogo normalizado de cartas y sets**, independiente de las reglas de colección del usuario. El usuario puede definir qué cartas considera pertenecientes a cada uno de sus sets, registrar las cartas físicas que posee, consultar el progreso de completitud, almacenar fotografías propias y estimar el valor económico de la colección mediante precios externos.

Stack

- **Backend:** Flask + Python
 - **Frontend:** Vanilla JS (SPA), CSS custom
 - **Persistencia:** SQLite
 - **Acceso a datos:** [PokemonRepo](#)
 - **Imágenes:** filesystem local
 - **Precios:** integración con fuente externa de precios (Cardmarket u otra API compatible)
 - **Deploy:** Docker + docker-compose
 - **Uso:** aplicación personal, un único usuario
 - **Acceso:** desktop y mobile
-

Funcionalidades

1. Catálogo de cartas

La aplicación dispone de un catálogo maestro de cartas Pokémon.

Cada carta del catálogo representa una **carta lógica**, no necesariamente una variante física concreta.

Ejemplo:

```
Base Set
└─ Charizard #4
```

Las variantes como:

- Holo / non-holo
- Reprints

- Unlimited / 2nd Edition
- Otras variantes

pueden existir en las fuentes externas, pero no necesariamente representan cartas diferentes dentro del catálogo personal.

El catálogo debe contener, como mínimo:

- ID interno
 - Nombre
 - Set oficial
 - Número de carta
 - Rareza
 - Tipo
 - Imagen de referencia
 - Identificador/es externo/s para APIs de precios
 - Información necesaria para relacionar la carta con variantes oficiales
-

2. Sets personalizados

La aplicación debe permitir definir una **estructura de colección propia**, independiente de la estructura oficial del catálogo.

Un set personal puede:

- Corresponder exactamente a un set oficial.
- Agrupar varios sets oficiales.
- Ser un subconjunto de un set oficial.
- Excluir determinadas cartas.
- Ignorar determinadas variantes.
- Agrupar reprints como una misma carta lógica.

Ejemplo:

```
Mi colección
├─ Base Set
│   ├── Bulbasaur
│   ├── Ivysaur
│   ├── Venusaur
│   └─ ...
└─ Charizard
```

Aunque oficialmente existan múltiples variantes/reprints de esas cartas, el usuario puede decidir que todas cuentan como:

Base Set → Charizard

Los sets personalizados deben poder definir explícitamente qué cartas forman parte de ellos.

El listado de sets personalizados es el siguiente:

- Gen1
 - Base Set
 - Jungle (sin holos)
 - Fossil (sin holos)
 - Team rocket (sin holos)
 - Gym Heroes (sin holos)
 - Gym Challenge (sin holos)
 - WOTC Promos
 - Gen 2
 - Neo Genesis (sin holos)
 - Neo Discovery (Sin holos)
 - Neo reveletation (sin holos)
 - Neo destiny (sin holos)
 - Gen 3
 - EX Team rocket Returns (sin comunes)
-

3. Vista completa de un set

Debe existir una vista visual que muestre **todas las cartas que forman parte del set personalizado**, independientemente de si el usuario las posee.

Ejemplo:

BASE SET

67 / 102 cartas

65,7 % completado

[Bulbasaur] [Ivysaur] [Venusaur] [Charmander]

[Charmeleon] [????????] [Blastoise] [Charizard]

...

Cartas poseídas

Se muestra la imagen de la carta y su información.

Cartas no poseídas

Se muestran mediante un placeholder visual.

El usuario debe poder identificar fácilmente:

- Qué cartas posee.
- Qué cartas faltan.
- Número de carta.
- Progreso del set.

La vista debe permitir ordenar/filtrar las cartas según criterios relevantes, como:

- Número
 - Nombre
 - Rareza
 - Poseída / faltante
-

4. Vista "Mi colección"

Debe existir una segunda vista que muestre exclusivamente las cartas que el usuario posee.

Esta vista representa el **inventario real de la colección**, no el catálogo completo.

Debe permitir:

- Buscar cartas.
- Filtrar por set.
- Filtrar por condición.
- Filtrar por variante.
- Filtrar por idioma.
- Filtrar por rareza.
- Ordenar por diferentes criterios.
- Ver cantidad de copias.
- Ver fotografía propia cuando exista.
- Ver valor estimado.

Debe distinguirse entre:

Cartas únicas

Ejemplo:

67 cartas diferentes

Cartas físicas

Ejemplo:

94 cartas físicas

Si el usuario tiene:

Charizard ×3

cuenta como:

- 1 carta completada dentro del set.
 - 3 cartas físicas dentro del inventario.
-

5. Registro de cartas

Desde la vista de un set o desde la colección se debe poder abrir un modal para registrar o editar una carta.

El modal debe permitir:

- Seleccionar variante.
- Seleccionar condición.
- Seleccionar idioma.
- Indicar cantidad.
- Subir fotografía → Opcional. Si no se sube. Ponemos un place holder con asi: Base Set #4. Carta fondo gris y letras en negro o blanco lo que quede mejor.
- Editar información previamente registrada.
- Guardar cambios.
- Eliminar el registro.

Ejemplo:

Charizard

Base Set #4

Variante:

[Holo ▼]

Condición:

[NM] [LP] [MP] [HP] [Damaged]

Idioma:

[Español ▼]

Cantidad:

[1]

Fotografía:

[Subir imagen]

[GUARDAR]

6. Variantes

Las variantes se registran en el inventario del usuario, pero **no necesariamente generan una carta lógica diferente dentro de la colección.**

Por ejemplo:

Base Set / Charizard

puede tener:

Holo

Non-Holo

sin que eso cambie el progreso de completitud.

La aplicación debe poder conservar la variante física porque puede ser relevante para:

- Identificación.
- Valoración.
- Inventario.
- Precio.
- Fotografía.

Pero la lógica de completitud debe depender de la **carta lógica del catálogo personalizado**.

En 1 frase. Ponele que tenemos 1 gengar foil y no foil en la coleccion de fossil. El modal tiene que ser capaz de agregar 1 foil y luego 1 no foil. El progreso de la coleccion se mantiene igual por que ya con cualquiera de los dos como no contamos foils para fossil da igual. Luego para estimar el valor, sumamos cada variante. Luego tendríamos que ser capaces de cargar 1 imagen por cada variante. y mostramos por default la foto en mejor estado en el grid view. Cuando entramos en el modal de la carta, vemos todas las variantes que en movil se puedan ver haciendo swipe.

7. Condición

Cada registro de colección debe poder tener una condición.

Valores iniciales:

- NM — Near Mint
- LP — Lightly Played
- MP — Moderately Played
- HP — Heavily Played
- Damaged

La condición debe poder utilizarse posteriormente para estimar el valor de la carta.

8. Idioma

Cada registro físico debe poder registrar su idioma.

Como mínimo:

- Español
- Inglés
- Portugués
- Otros

El idioma no debe modificar necesariamente la identidad lógica de la carta.

9. Fotografías propias

El usuario puede subir una fotografía propia de la carta desde desktop o móvil.

La aplicación debe:

- Permitir seleccionar una imagen desde el dispositivo.
- Permitir utilizar directamente la cámara del móvil cuando el navegador lo permita.
- Asociar la fotografía con el registro de colección.
- Mostrar la fotografía en la vista de colección.
- Mostrarla en el modal de edición.
- Permitir reemplazarla.
- Permitir eliminarla.

Las imágenes se almacenan en filesystem, no dentro de SQLite.

Ejemplo:

```
data/  
└─ images/  
    ├── collection_001.jpg  
    ├── collection_002.jpg  
    └─ ...
```

No se implementará reconocimiento OCR ni reconocimiento automático de cartas mediante visión.

La identificación de la carta se realiza manualmente desde el catálogo.

10. Valoración de la colección

La aplicación debe poder estimar el valor económico de las cartas mediante una fuente externa de precios.

La fuente inicial será preferentemente:

- Cardmarket
- u otra API/fuente de precios compatible

La aplicación debe asociar cada carta lógica con uno o varios identificadores externos necesarios para consultar precios.

11. Precios por carta

El precio debe obtenerse independientemente del inventario.

Si el usuario posee:

Charizard ×3

el precio se consulta **una sola vez** para esa carta/variante relevante y luego se multiplica por la cantidad.

No debe realizarse una consulta externa por cada copia física.

12. Cache de precios

Los precios externos no se deben consultar cada vez que se abre una pantalla.

La aplicación debe mantener un cache local de precios → Diría que esto hay que fetchearlo 1 vez por mes con un cron.

Cada precio debe almacenar como mínimo:

- Carta
- Variante/fuente utilizada
- Fuente
- Precio
- Fecha/hora de actualización

Ejemplo:

```
Charizard
Base Set
Holo
Cardmarket
€185.00
2026-08-25
```

La aplicación debe poder determinar si un precio está suficientemente actualizado antes de volver a consultarlo.

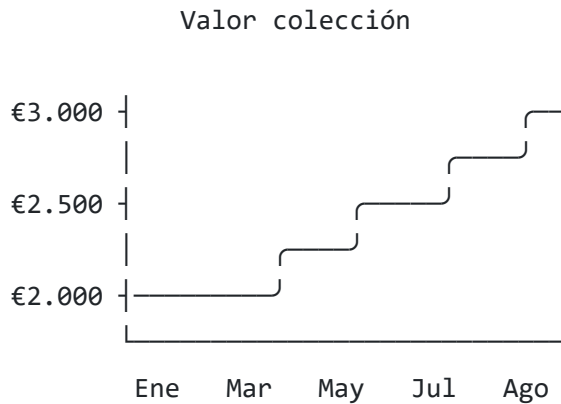
13. Historial de precios

La aplicación debe conservar snapshots históricos de los precios.

Esto permitirá visualizar la evolución temporal del valor de:

- Una carta.
- Un set.
- Toda la colección.

Ejemplo:



14. Estimación del valor de una carta

El valor estimado debe tener en cuenta, cuando sea posible:

- Carta.
- Variante.
- Condición.
- Idioma.
- Fuente de precios.

Si la fuente externa no dispone de un precio exacto para una combinación determinada, debe existir una estrategia de fallback configurable.

Ejemplo conceptual:

```
precio exacto
↓
precio de variante compatible
↓
precio general de la carta
```

15. Valor total de la colección

La aplicación debe mostrar:

- Valor estimado total en euros.
- Valor por set.
- Valor por carta.
- Valor según cantidad.
- Evolución histórica del valor.

Ejemplo:

Mi colección

1.247 cartas físicas

842 cartas únicas

68 sets

73,4 % completitud

Valor estimado

€3.842

16. Dashboard

La aplicación debe disponer de un dashboard general con estadísticas de la colección.

Como mínimo:

- Número de cartas únicas.
 - Número total de cartas físicas.
 - Número de sets.
 - Porcentaje global de completitud.
 - Valor estimado de la colección.
 - Valor por set.
 - Sets más completos.
 - Sets con más cartas faltantes.
 - Cartas de mayor valor.
 - Evolución del valor de la colección.
-

17. Progreso de colección

El progreso debe calcularse sobre las **cartas lógicas definidas por los sets personalizados**, no sobre el número de variantes oficiales.

Ejemplo:

Base Set

102 cartas objetivo

78 cartas poseídas

Completitud: 76,5 %

Tener:

Charizard Holo

Charizard Non-Holo

sigue contando como:

Charizard = completado

una sola vez.

18. Cartas faltantes

La aplicación debe permitir visualizar rápidamente las cartas faltantes de un set.

Ejemplo:

BASE SET

Faltan 24 cartas

#05 Charmeleon

#09 Blastoise

#12 Ninetales

#17 Pidgeotto

...

Las cartas faltantes deben poder ordenarse por:

- Número.
- Nombre.
- Rareza.
- Valor estimado.

Esto permitirá utilizar la aplicación también como **wishlist de cartas pendientes de conseguir**.

19. Búsqueda global

Debe existir una búsqueda global de cartas.

Debe poder encontrar una carta por:

- Nombre.
- Número.
- Set.
- ID.
- Otros campos relevantes.

La búsqueda debe funcionar tanto sobre:

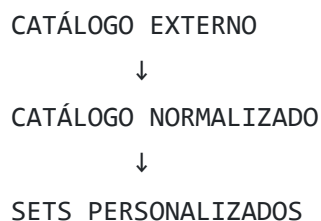
- Catálogo.
 - Colección.
-

20. Administración del catálogo

Debe existir una forma de actualizar/importar el catálogo maestro desde fuentes externas o archivos de datos.

La información externa no debe sobrescribir directamente la información específica de la colección del usuario.

Debe mantenerse la separación:





COLECCIÓN DEL USUARIO

21. Separación entre catálogo y colección

Esta separación es fundamental.

Catálogo

Describe qué cartas existen.

Set personalizado

Describe qué cartas considera el usuario pertenecientes a un determinado conjunto.

Colección

Describe qué cartas físicas posee el usuario.

Precio

Describe cuánto vale aproximadamente una determinada carta/variante en una fuente externa.

Una modificación del catálogo externo no debe destruir los registros de colección existentes.

22. Persistencia

La persistencia se realizará mediante SQLite.

La aplicación debe centralizar todo acceso a datos mediante:

PokemonRepo

Los endpoints Flask no deben realizar consultas SQL directamente.

La estructura conceptual será:

Flask routes



PokemonRepo



SQLite

Las imágenes permanecerán fuera de SQLite en filesystem.

23. API REST

La API deberá cubrir, como mínimo, las siguientes áreas:

Catálogo

GET	/api/cards	Lista/búsqueda de cartas
GET	/api/cards/<id>	Detalle de carta
Método	Endpoint	Descripción

Sets

GET	/api/sets	Lista de sets personalizados
GET	/api/sets/<id>	Detalle de set
Método	Endpoint	Descripción
PUT	/api/sets/<id>	Actualizar set
DELETE	/api/sets/<id>	Borrar set

Colección

GET	/api/collection	Lista de cartas poseídas
POST	/api/collection	Añadir carta
Método	Endpoint	Descripción
DELETE	/api/collection/<id>	Eliminar carta

Imágenes

--	--	--

Método	Endpoint	Descripción
DELETE	/api/collection/<id>/image	Eliminar fotografía

Precios

Método	Endpoint	Descripción
GET	/api/prices/<card_id>	Obtener precio/cache
POST	/api/prices/update	Actualizar precios

Dashboard

Método	Endpoint	Descripción
--------	----------	-------------

24. Estructura de datos conceptual

La base SQLite deberá separar, como mínimo:

- cards
- sets
- set_cards
- collection
- price_cache
- price_history

cards

Catálogo normalizado de cartas.

Campo	Tipo	Notas
id	str	PK
name	str	

set_id	str	Set lógico principal
number	str	
rarity	str	
image	str	Imagen de catálogo
external_ids	JSON/text	IDs de fuentes externas

sets

Sets personalizados del usuario.

id	str	PK
name Campo -----	str Tipo -----	Notas -----
order	int	orden visual

set_cards

Relación entre un set personalizado y las cartas que lo componen.

set_id	str	FK
card id Campo -----	str Tipo -----	FK Notas -----

Esta tabla permite que un set sea un subconjunto o agrupación arbitraria del catálogo.

collection

Inventario del usuario.

id	int	PK
card id Campo -----	str Tipo -----	FK Notas -----

condition	str	NM/LP/MP/HP/Damaged
language	str	idioma
quantity	int	número de copias
image	str	fotografía propia
notes	str	opcional

price_cache

Precio actualmente conocido.

card_id	str	FK
variant	str	variante relevante
Campo	Tipo	Notas
price	float	
currency	str	
updated_at	datetime	

price_history

Snapshots históricos.

id	int	PK
card id	str	FK
Campo	Tipo	Notas
source	str	
price	float	
currency	str	
timestamp	datetime	

25. Imágenes

Estructura conceptual:

```
pokemon/

├─ app.py
├─ requirements.txt
├─ Dockerfile
├─ docker-compose.yml
├─
├─ api/
│   └─ routes.py
├─
├─ services/
│   └─ repository.py
├─
├─ data/
│   └─ pokemon.db
├─
├─ media/
│   └─ collection/
│       ├── 001.jpg
│       ├── 002.jpg
│       └─ ...
├─
├─ static/
│   ├── style.css
│   └─ pokemon.js
├─
└─ templates/
    └─ index.html
```

Las imágenes de catálogo pueden almacenarse separadamente de las fotografías propias de la colección.

26. Frontend

La aplicación será una SPA en Vanilla JS.

La navegación principal deberá contemplar, como mínimo:

Dashboard
Collection
Sets
Wishlist / Missing

La interfaz debe estar diseñada para funcionar correctamente tanto en:

- Desktop.
- Mobile.

El móvil debe ser un caso de uso prioritario para la carga de fotografías.

27. Visualización de cartas

Las cartas deben mostrarse principalmente mediante una interfaz de tipo **grid/gallery**.

Cada card visual puede mostrar:

- Imagen.
- Número.
- Nombre.
- Indicador de posesión.
- Cantidad.
- Condición.
- Valor estimado.

Las cartas faltantes deben tener un tratamiento visual claramente diferenciado mediante placeholders.

28. Modal de carta

El modal será el punto central de interacción con una carta.

Desde él se debe poder:

- Consultar información de catálogo.
- Ver imagen de referencia.
- Ver estado de colección.
- Añadir carta.
- Editar cantidad.
- Editar variante.
- Editar condición.

- Editar idioma.
- Subir/reemplazar fotografía.
- Ver valor estimado.
- Ver información de precio.
- Eliminar el registro.

No habrá OCR ni reconocimiento automático de cartas.

29. Historial de la colección

La aplicación debe conservar información suficiente para representar la evolución de la colección a lo largo del tiempo.

Debe ser posible representar:

- Evolución del número de cartas.
- Evolución de la completitud.
- Evolución del valor estimado.
- Evolución del número de sets completados.

Los eventos históricos pueden derivarse de snapshots periódicos y/o del historial de precios.

30. Automatización de precios

La actualización de precios debe poder ejecutarse automáticamente mediante un proceso programado.

Conceptualmente:

```
graph TD
    A[Scheduled job] --> B[buscar cartas de la colección]
    B --> C[comprobar antigüedad del cache]
    C --> D[consultar fuente externa cuando sea necesario]
    D --> E[guardar precio actual]
    E --> F[guardar snapshot histórico]
```

No se deben consultar precios de cartas que no sean relevantes para la colección salvo que sea necesario para otras funcionalidades.

31. Principios de diseño

- La aplicación es personal y de un único usuario.
- El estilo tiene que ser minimalista y moderno.
- No requiere autenticación.
- La colección del usuario es la fuente de verdad sobre las cartas poseídas.
- El catálogo externo no define obligatoriamente la estructura de colección.
- Las variantes físicas pueden existir sin generar una carta adicional para la completitud.
- Los sets personalizados pueden ser subconjuntos o agrupaciones arbitrarias.
- Las fotografías son opcionales.
- No se implementará OCR.
- Los precios externos son estimaciones y no constituyen una valoración garantizada.
- El acceso a SQLite debe estar centralizado en [PokemonRepo](#).
- Las imágenes se almacenan en filesystem.
- La aplicación debe ser usable cómodamente desde móvil.
- La interfaz debe priorizar la visualización gráfica de las cartas y el progreso de la colección.